A comprehensive WebRTC walkthrough

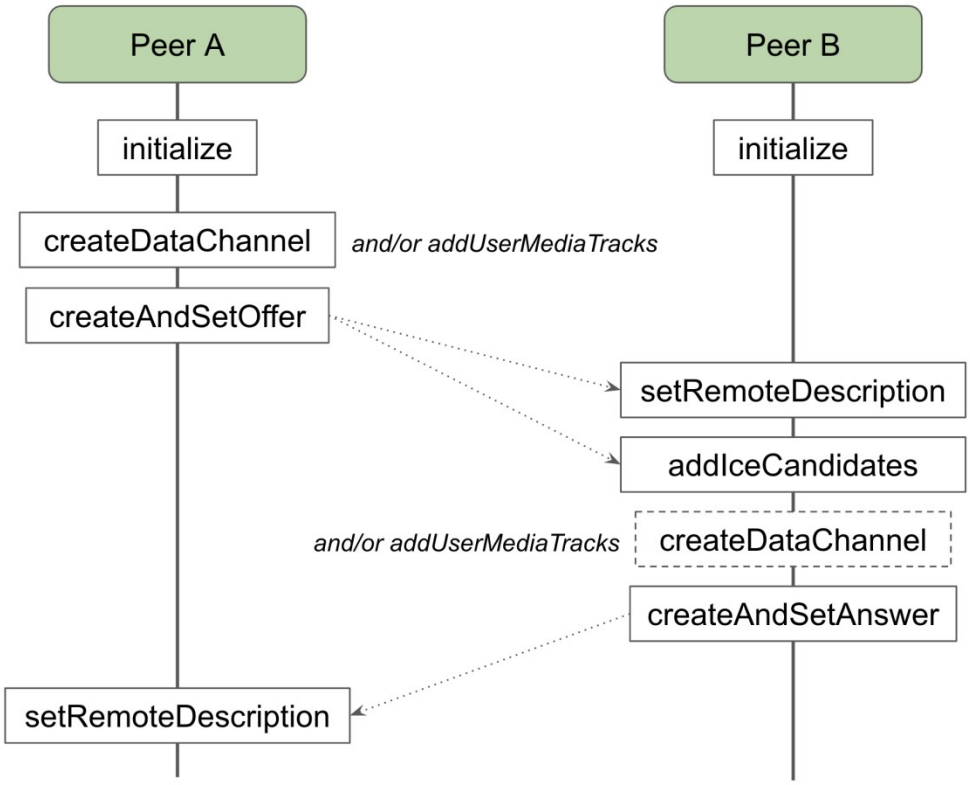
2023-07-12 9 mins **en**

WebRTC allows for real-time communication between two peers using only the browser's built-in functionalities, with no need for a communications server. That's AWESOME. But turns out that the browser API is complex, and I find the official samples repository a bit confusing. Here is an attempt to provide a clearer example.

The idea is to establish a connection between two browsers (or two tabs of the same browser) running the same Javascript code. The browsers, or **peers**, can be either in the same device or in different devices and each of them will be able to act both as the **starter** or the **receiver** of a connection.

Connection overview

Before diving into the code it's worth understanding the steps involved in establishing a WebRTC connection. Let's have a look at the following simplified connection diagram (based on the WebRTC Connectivity documentation), which reflects the basic operations the two peers need to execute:



WebRTC connection diagram

1. **Initialize.** Both peers create a new connection object.
2. **Media.** Peer A adds media to the connection (i.e. data channels and/or stream tracks).

3. **Offer creation**. Peer A creates an offer (i.e. session description) and sets it as the local description. The latter will generate several ICE candidates.

4. **Offer exchange**. Peer A sends the offer and its ICE candidates to peer B through the **signaling service**.

5. **Offer reception**. Peer B sets the offer as the remote description and then adds the remote ICE candidates.

6. **Media**. If needed, Peer B adds media to the connection (i.e. data channels and/or stream tracks).

7. **Answer creation**. Peer B creates an answer (i.e. session description) and sets it as the local description. Again, the latter will generate several ICE candidates.

8. **Answer exchange**. Peer B sends the answer to peer A through the **signaling service**. This time, it's not necessary to send the ICE candidates.

9. **Answer reception**. Peer A sets the answer as the remote description. The connection has been established!

The steps 4 and 8 involve a so called "signaling service". In communication systems (e.g. post, telephone, email, instant messaging, etc.) when a peer wants to send a message or establish a connection they need to know the identifier of the recipient: a postal address, a phone number, an email address, etc.

In WebRTC however peers do not have identifiers. Instead each peer generates two pieces of information every time they want to establish a new connection, and both pieces are valid only once. From the MDN documentation:

- **Session description**: includes information about the kind of media being sent, its format, the transfer protocol being used, the endpoint's IP address and port, and other information needed to describe a media transfer endpoint. It looks like this:


```
{"type":"offer","sdp":"v=0\r\no=- 8446939022420648928 2 IN IP4
127.0.0.1\r\ns=-\r\nt=0 0\r\na=group:BUNDLE 0\r\na=extmap-allow-
mixed\r\na=msid-semantic: WMS\r\nm=application 9 UDP/DTLS/SCTP webrtc-
datachannel\r\nc=IN IP4 0.0.0.0\r\na=ice-ufrag:RdRl\r\na=ice-
pwd:EkNvZFozr3G6cCQcblkfYWI9\r\na=ice-
options:trickle\r\na=fingerprint:sha-256
76:53:B4:D3:73:BD:B7:AE:61:7F:05:33:61:34:85:F7:3C:68:05:EC:93:BE:F8:0A:
FD:BB:E3:4D:83:1A:B5:50\r\na=setup:actpass\r\na=mid:0\r\na=sctp-
port:5000\r\na=max-message-size:262144\r\n"}
```


- **ICE Candidate**: includes information about the network connection and details the available methods the peer is able to communicate through. It looks like this:


```
{"candidate":"candidate:3426902834 1 udp 2113939711 60c8b1aa-d1e7-
46f7-954d-9183cc7efe63.local 54523 typ host generation 0 ufrag RdRl
```

```
network-cost
999","sdpMid":"0","sdpMLineIndex":0,"usernameFragment":"RdRl"}
```

Because this information needs to be acquired by each peer PRIOR to establishing the connection, we need to exchange it via an already existing connection or channel. Such channel is called **signaling service** and it can be anything: from an actual server (most likely a websocket server) to, quoting the documentation, "email, postcard, or a carrier pigeon".

In this example we will use the computer's clipboard as our signaling service: in order to exchange the session data we will copy it from one browser and paste it in the other 📋

Coding time

Based on the steps described above we need to implement the following functions:

- `initialize`. Creates a new RTCPeerConnection instance and defines a bunch of event handlers:
  - `onicecandidate` (connection). Will be called for each local ICE candidate generated when setting the local description. We will need to capture those candidates in order to send them to the other peer.
  - `ontrack` (connection). Will be called for each stream track added by the other peer once the connection is established. We will want to consume those tracks, from an HTML video element for example.
  - `ondatachannel` (connection). Will be called for each data channel the other peer has created. We will need to keep track of those channels to receive/send messages through them.
  - `onopen` (channel). Will be called on a data channel once the connection is established. It tells us the channel is ready to start sending/receiving.
  - `onmessage` (channel). Will be called on a data channel every time a message is received. We will want to keep track of the messages and update the UI accordingly.
  - `onclose` (channel). Will be called on a data channel when the channel or the connection are closed. It tells us the channel is no longer available for sending/receiving.

```
export interface EventHandlers {
  onDataChannelClosed: () => void;
  onDataChannelOpened: (dataChannel: RTCDataChannel) => void;
  onIceCandidate: (candidate: RTCIceCandidate) => void;
  onMessageReceived: (message: string) => void;
  onRemoteStreamTrack: (track: MediaStreamTrack) => void;
```

```
}
export function initialize(eventHandlers: EventHandlers): RTCPeerConnection {
  const connection = new RTCPeerConnection();
  connection.onicecandidate = (event) => {
    /* Each event.candidate generated after creating the offer
    must be added by the peer answering the connection */
    if (event.candidate) {
      eventHandlers.onIceCandidate(event.candidate);
    }
  };
  /* This method will be called when the peer creates a channel */
  connection.ondatachannel = (event) => {
    const dataChannel = event.channel;
    dataChannel.onopen = () => {
      eventHandlers.onDataChannelOpened(dataChannel);
    };
    dataChannel.onmessage = (event) => {
      const message = event.data;
      eventHandlers.onMessageReceived(message);
    };
    dataChannel.onclose = () => {
      eventHandlers.onDataChannelClosed();
    };
  };
  /* This method will be called when the peer adds a stream track */
  connection.ontrack = (event) => {
    const { track } = event;
    eventHandlers.onRemoteStreamTrack(track);
  };
  return connection;
}
```
view rawrtc-peer-connection-in

The event handlers can be anything that works: plain functions, EventTarget, rxjs subscriptions, etc. Their implementation will depend on the application needs and the underlying framework. For a relatively simple React example, have a look at this file.

- `createDataChannel`. Creates a new RTCDataChannel object on a RTCPeerConnection object and defines a few event handlers (which we have already introduced in the previous bullet point):

```
export function createDataChannel(
  connection: RTCPeerConnection,
  label: string,
  eventHandlers: EventHandlers,
) {
  const dataChannel = connection.createDataChannel(label);
  dataChannel.onopen = () => {
    eventHandlers.onDataChannelOpened(dataChannel);
  };
```

```
  dataChannel.onmessage = (event) => {
    const message = event.data;
    eventHandlers.onMessageReceived(message);
  };
  dataChannel.onclose = () => {
    eventHandlers.onDataChannelClosed();
  };
}
```

- `addUserMediaTracks`. Adds media stream tracks (generated separately) to a RTCPeerConnection object.

```
export function addUserMediaTracks(
  connection: RTCPeerConnection,
  tracks: MediaStreamTrack[],
): RTCRtpSender[] {
  return tracks.map((track) => {
    return connection.addTrack(track);
  });
}
/* Usage: */
const mediaStream = await navigator.mediaDevices.getUserMedia({ /* ... */ });
addUserMediaTracks(connection, mediaStream.getTracks());
```

- `createAndSetOffer`. Creates an offer (i.e. session description) from a RTCPeerConnection object and sets the offer as the local description of the connection.

```
export async function createAndSetOffer(
  connection: RTCPeerConnection,
): Promise<RTCSessionDescriptionInit> {
  const offer = await connection.createOffer();
  await connection.setLocalDescription(offer);
  return offer;
}
```

- `createAndSetAnswer`. Creates an answer (i.e. session description) from a RTCPeerConnection object and sets the answer as the local description of the connection.

```
xport async function createAndSetAnswer(
  connection: RTCPeerConnection,
): Promise<RTCSessionDescriptionInit> {
  const answer = await connection.createAnswer();
  await connection.setLocalDescription(answer);
  return answer;
}
```

- `setRemoteDescription`. Sets the remote description of a RTCPeerConnection object, either the offer or the answer, received via signaling service.

```
export async function setRemoteDescription(
```

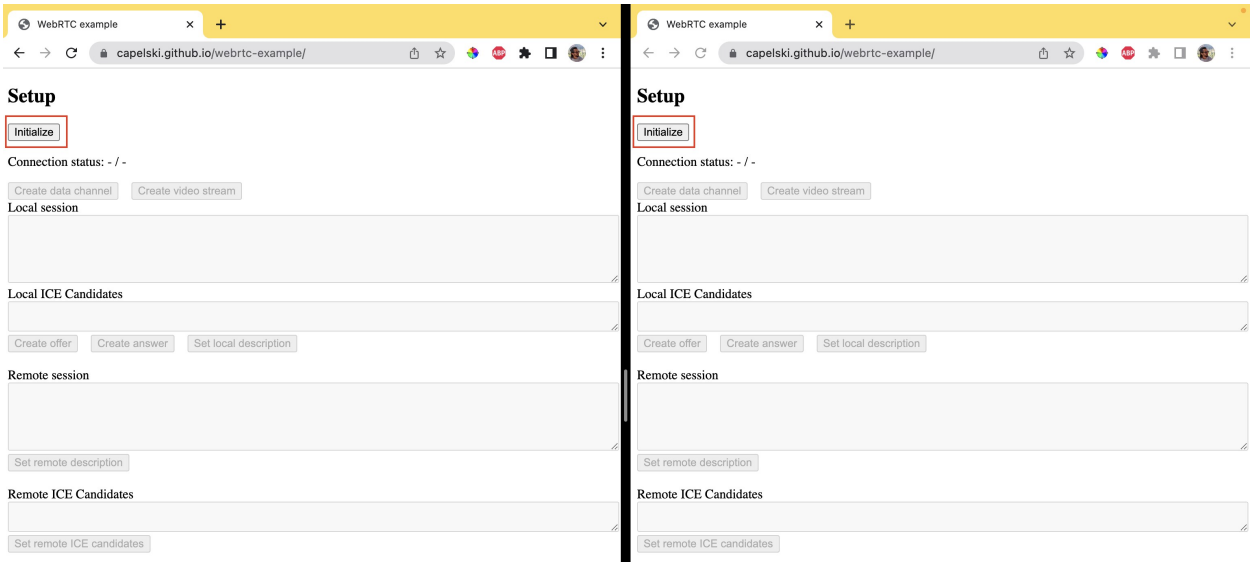
```
  connection: RTCPeerConnection,
  remoteDescription: RTCSessionDescriptionInit,
) {
  await connection.setRemoteDescription(remoteDescription);
}
```

- `addIceCandidates`. Adds the ICE candidates generated by the starting peer, received by the answering peer via signaling service.

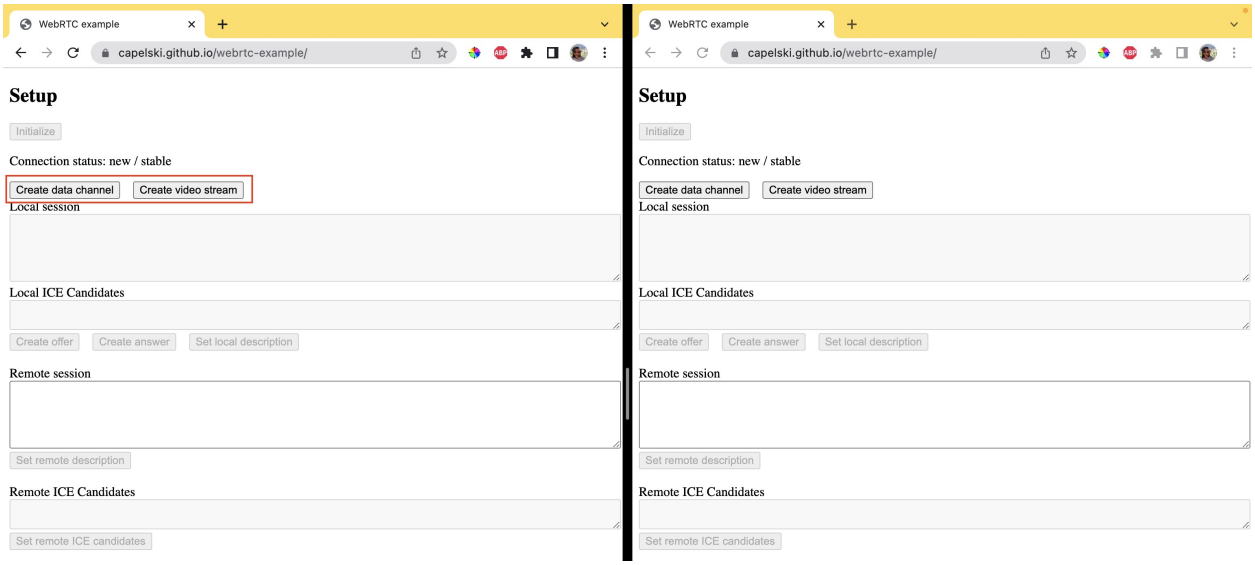
```
export async function addIceCandidates(
  connection: RTCPeerConnection,
  candidates: RTCIceCandidateInit[],
) {
  for (const candidate of candidates) {
    await connection.addIceCandidate(candidate);
  }
}
```

Here is a web app putting together all the functions: https://capelski.github.io/webrtc-example/. It is meant to reflect the connection negotiation, display the session description of each peer and help inputting the corresponding information of the other peer. It also has some logic to guarantee that the RTCPeerConnection methods are called in the right order.

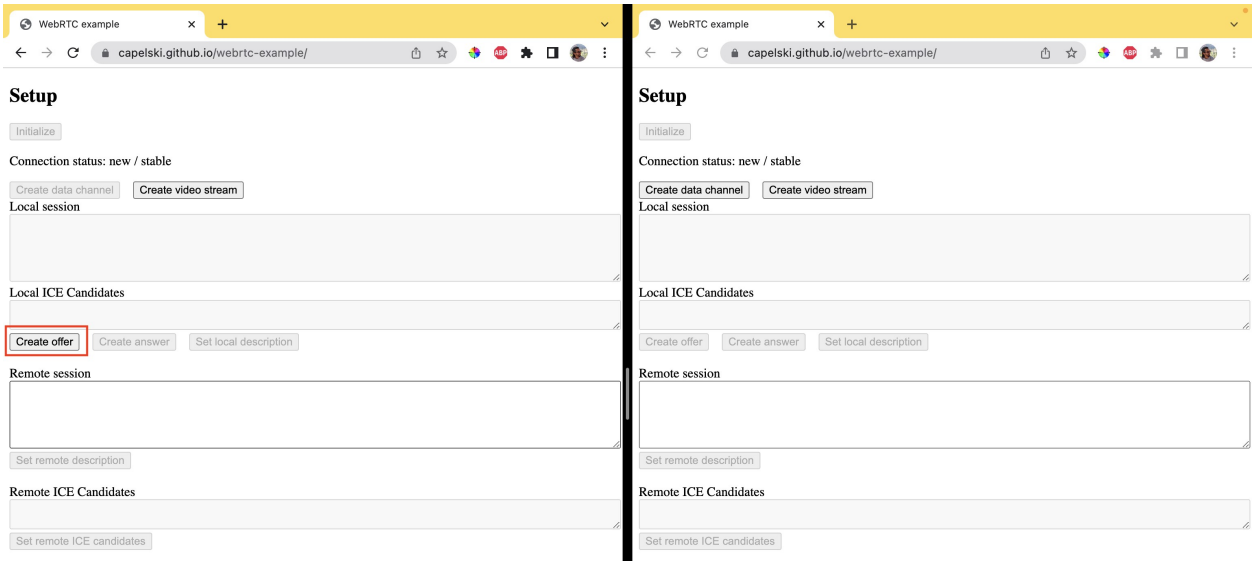
Note that, at the time of writing, the web app doesn't work on Firefox, since Firefox doesn't support connectionstate nor onconnectionstatechange.



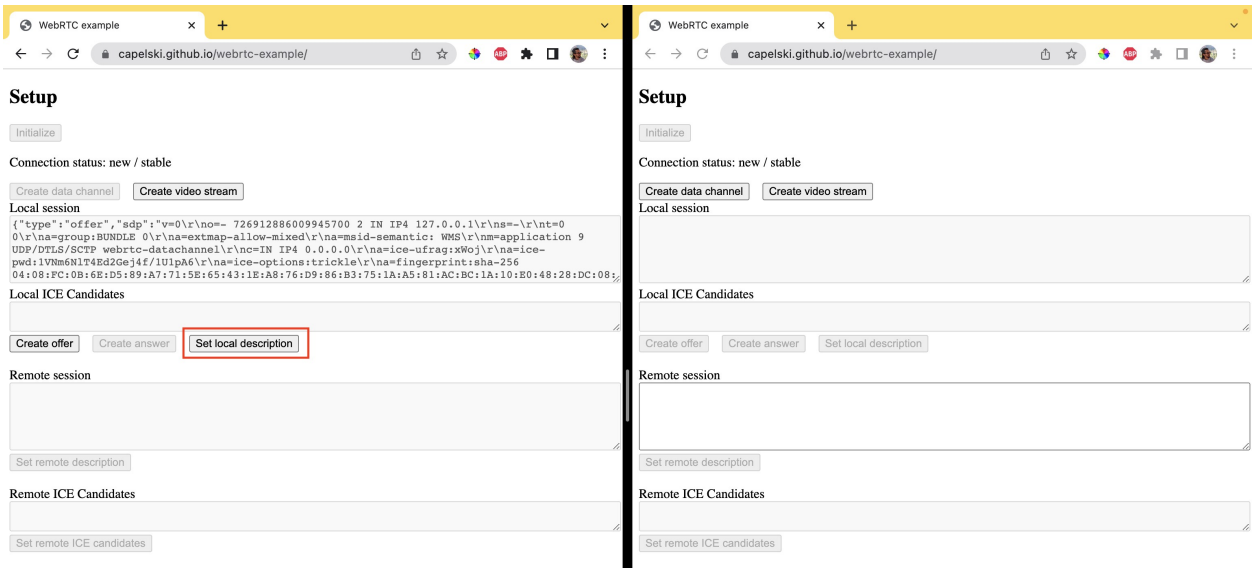
Initialize the RTCPeerConnection objects



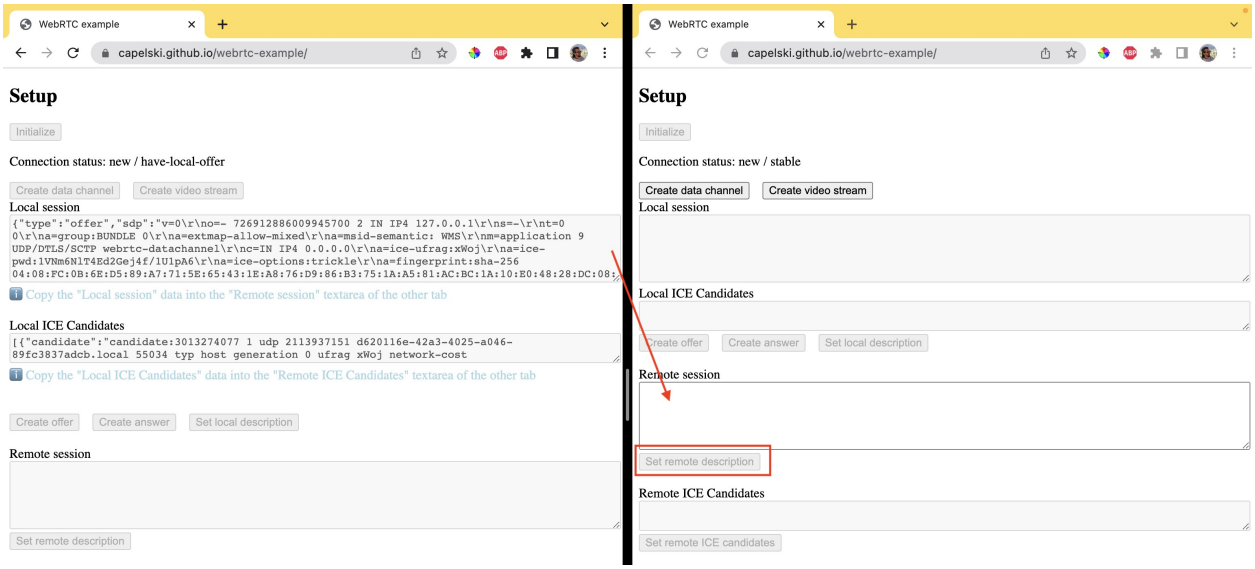
Create data channels and/or stream tracks in peer A



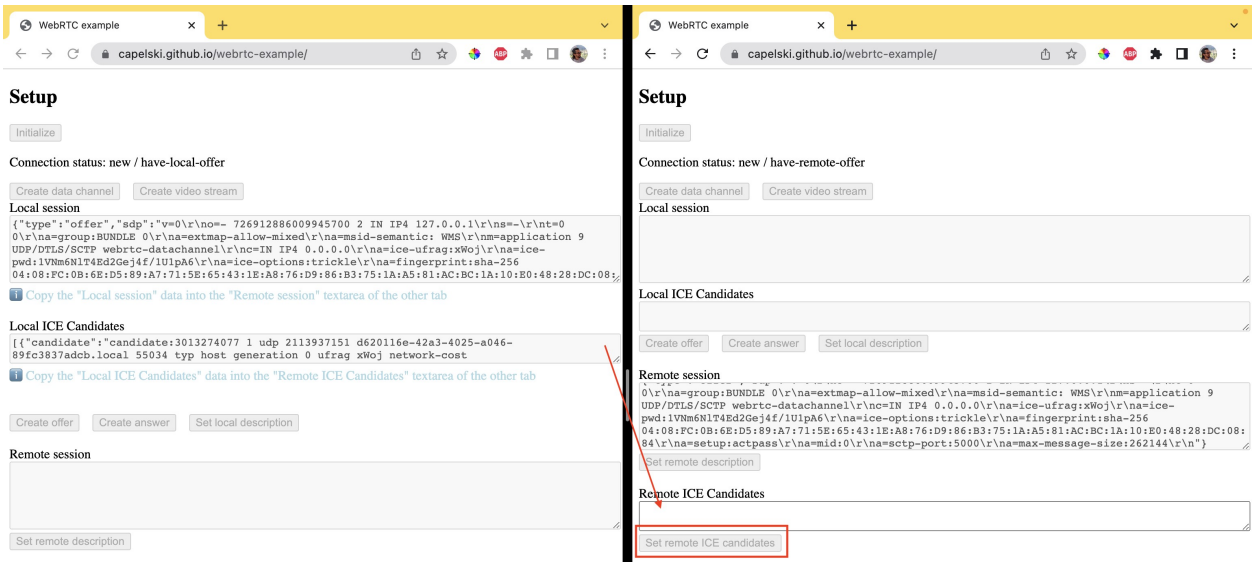
Create an offer in peer A



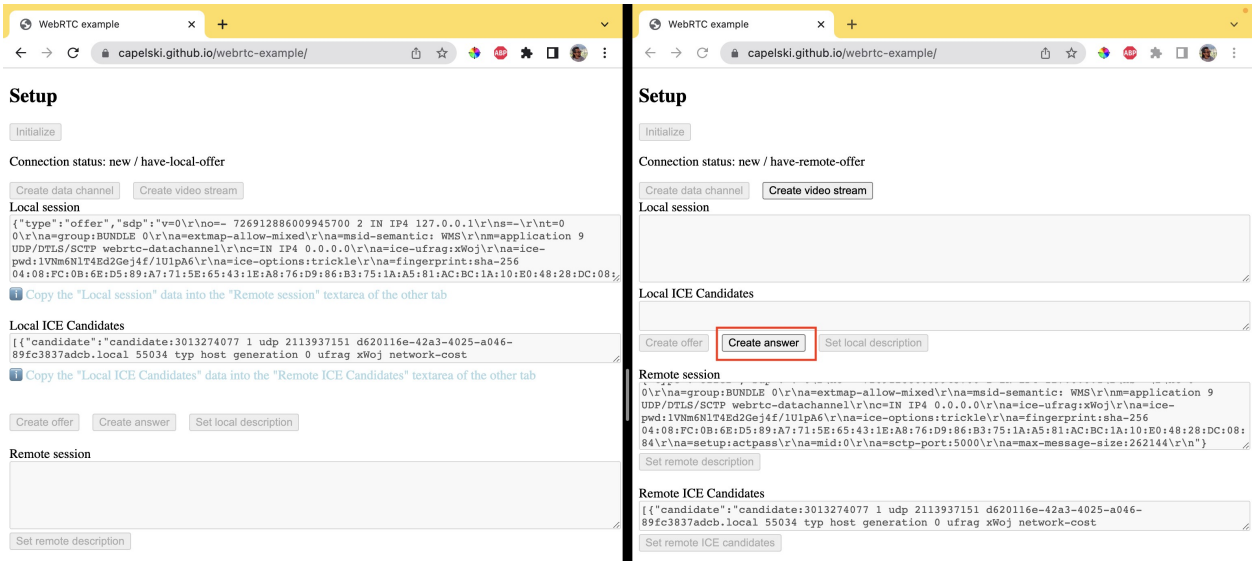
Set the offer as local description in peer A



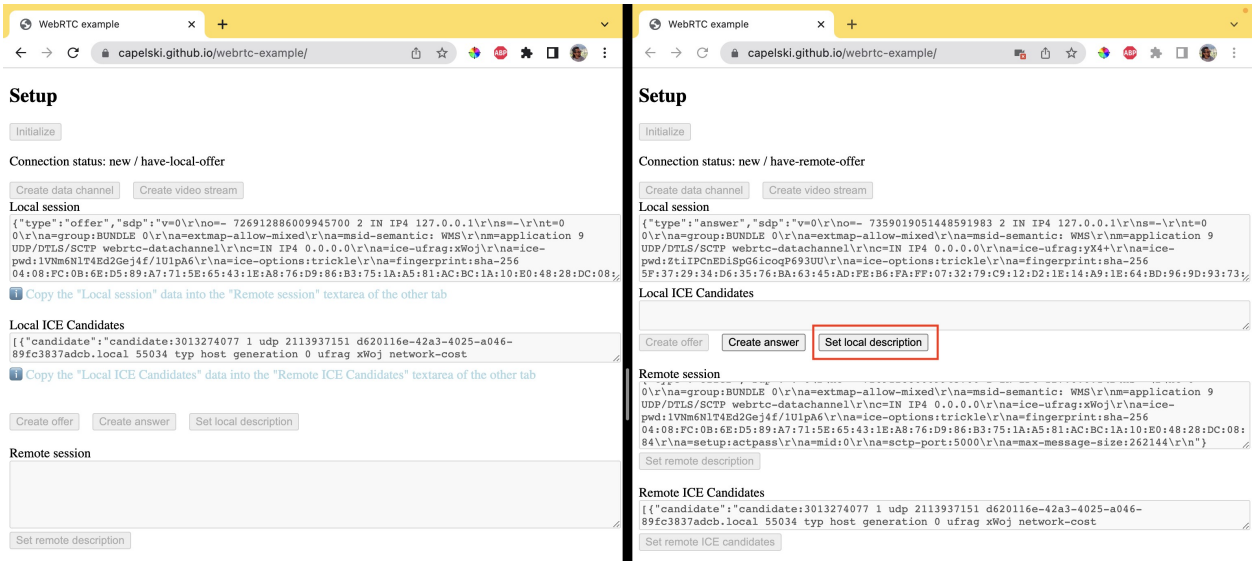
Set the offer as remote description in peer B



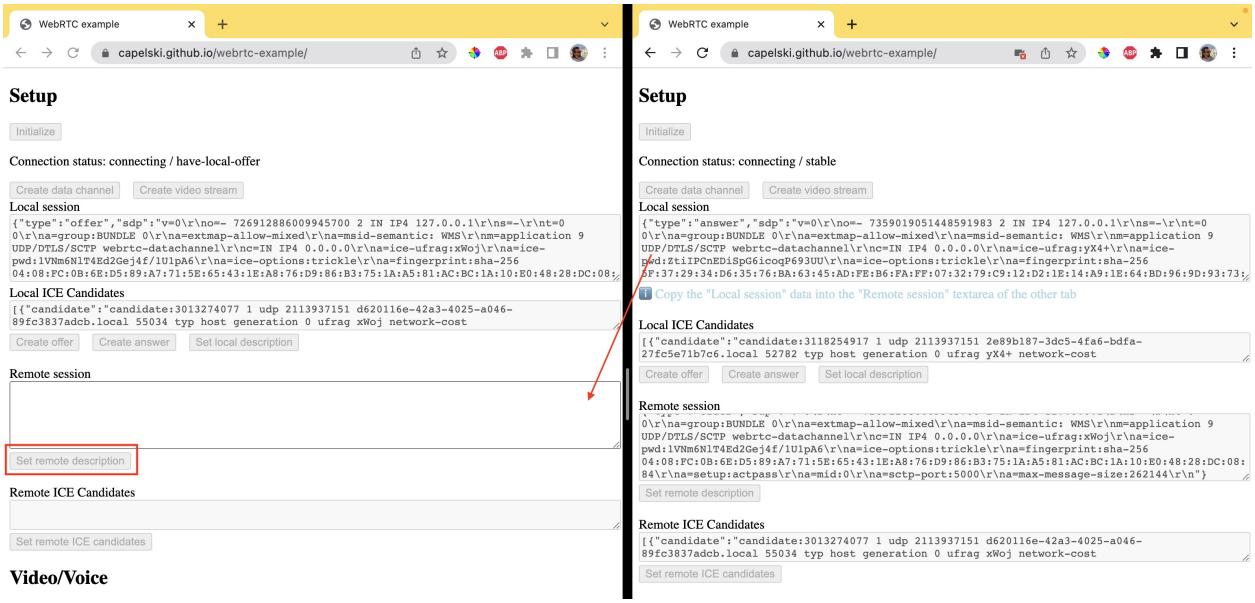
Add remote ICE candidates in peer B



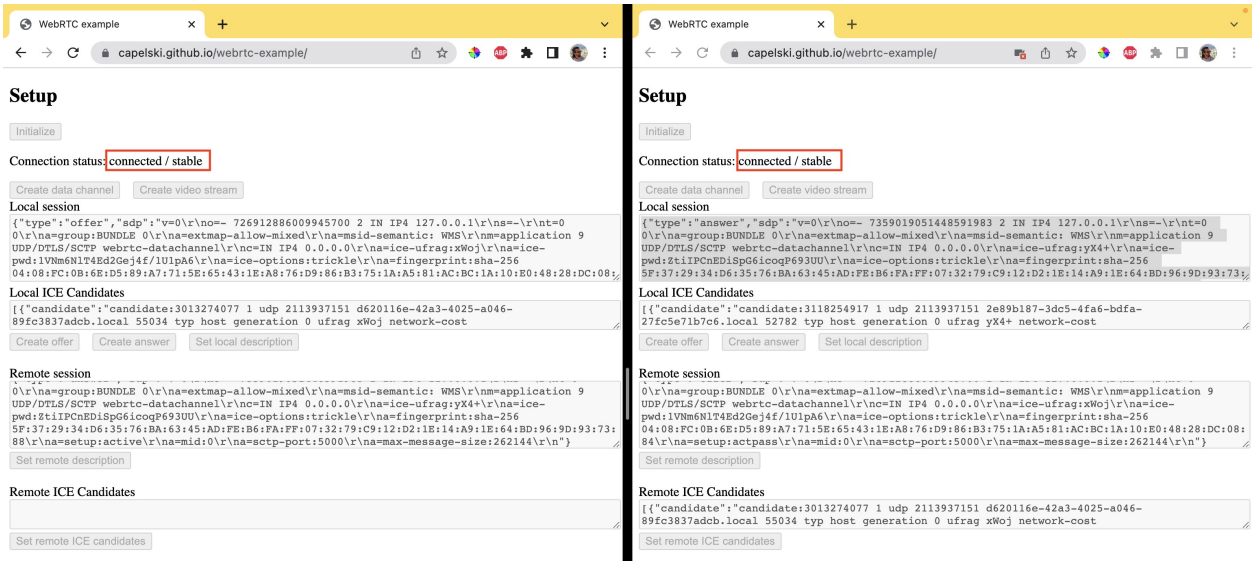
Create an answer in peer B



Set the answer as local description in peer B



Set the answer as remote description in peer A

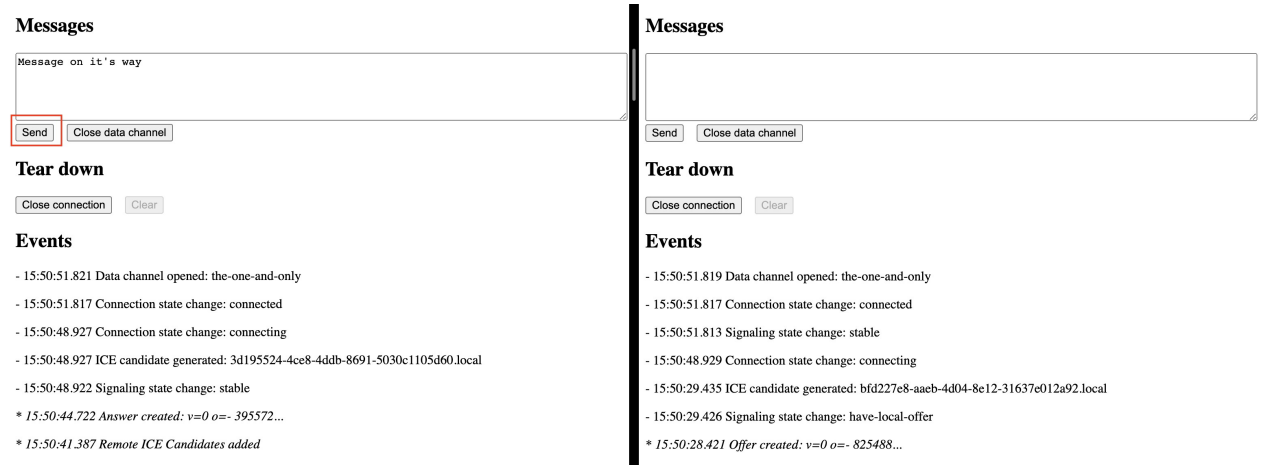


Connection negotiation finalized

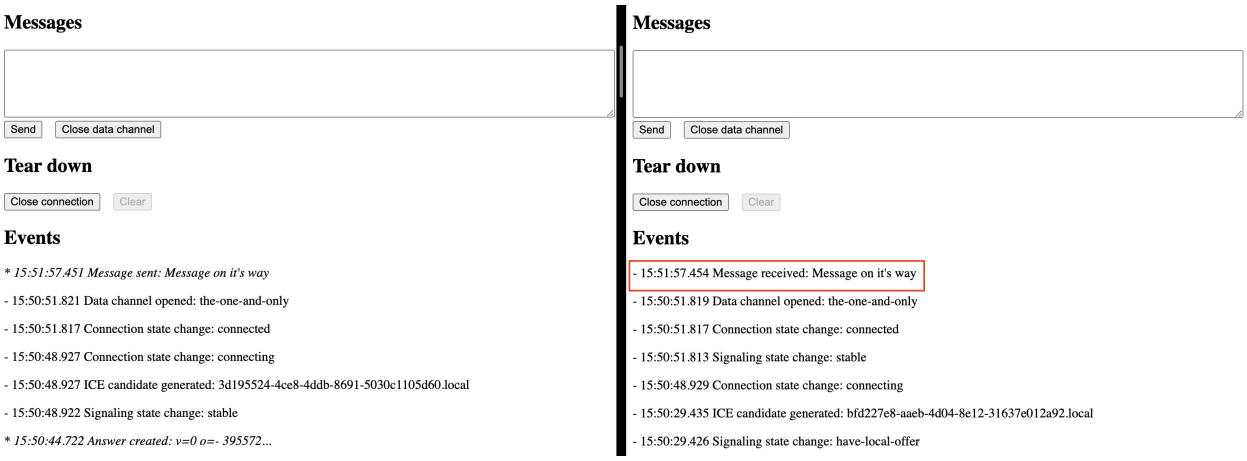
Data channels

After the connection has been established, every data channel triggers an `onopen` event, and both peers are able to send messages through them by calling the `send` method on the corresponding RTCDataChannel object. The incoming messages need to be handled through the `onmessage` handler (which we already defined earlier on).

```
export function sendMessage(dataChannel: RTCDataChannel, message: string) {
  dataChannel.send(message);
}
/* Message reception is handled via data channel handlers:
dataChannel.onmessage = (event) => {
  const message = event.data;
  // ...
};
*/
```

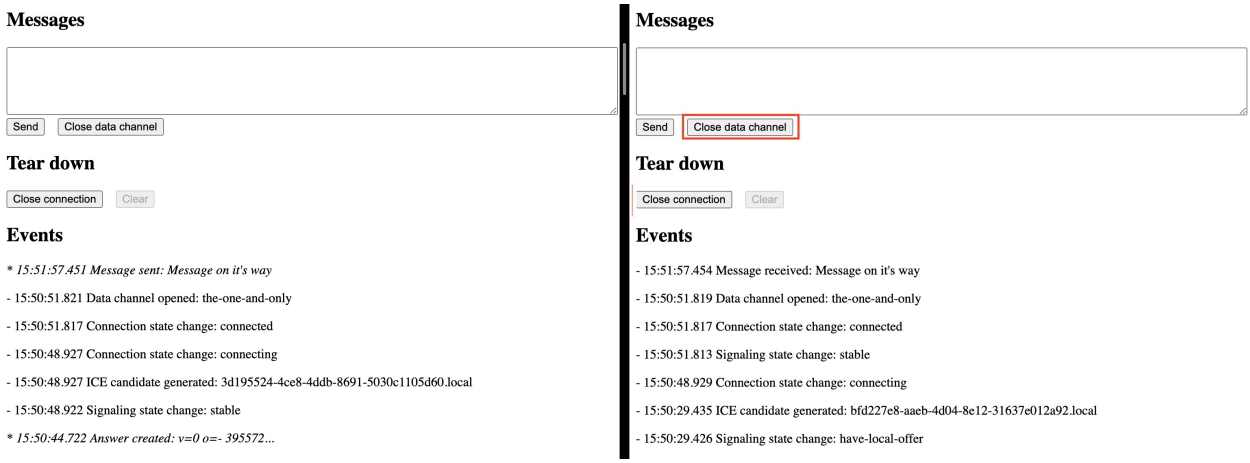


Sending message from peer A

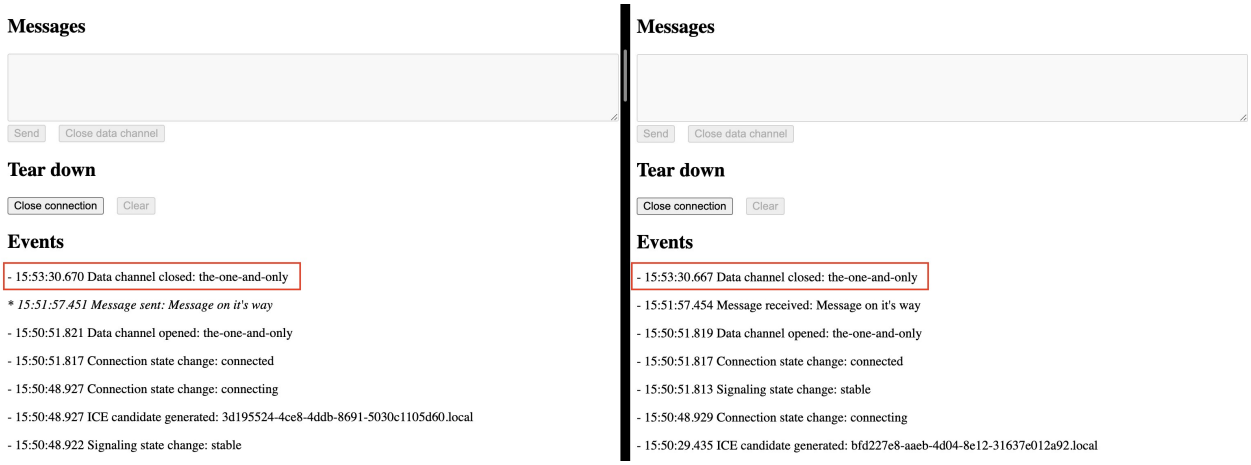


Message received by peer B

When a channel is no longer necessary it can be closed by any of the two peers by calling its `close` method. This will fire a `close` event on the RTCDataChannel object of both peers.



Closing data channel



Data channel closed

Stream tracks

After the connection has been established, remote stream tracks become available for the local peer to consume. Note that the stream tracks management is completely independent from the WebRTC connection. The connection object is only concerned about making the remote stream tracks available: creation, consumption and destruction must be handled explicitly. A few considerations:

- `addTrack` adds an existing track to the connection and returns a RTCRtpSender object, which can be used to remove the track from the connection.
- `removeTrack` removes a track from the connection using the corresponding RTCRtpSender object.
- Both `addTrack` and `removeTrack` won't have any effect once the connection has been established: they need to be called BEFORE generating an offer/answer.

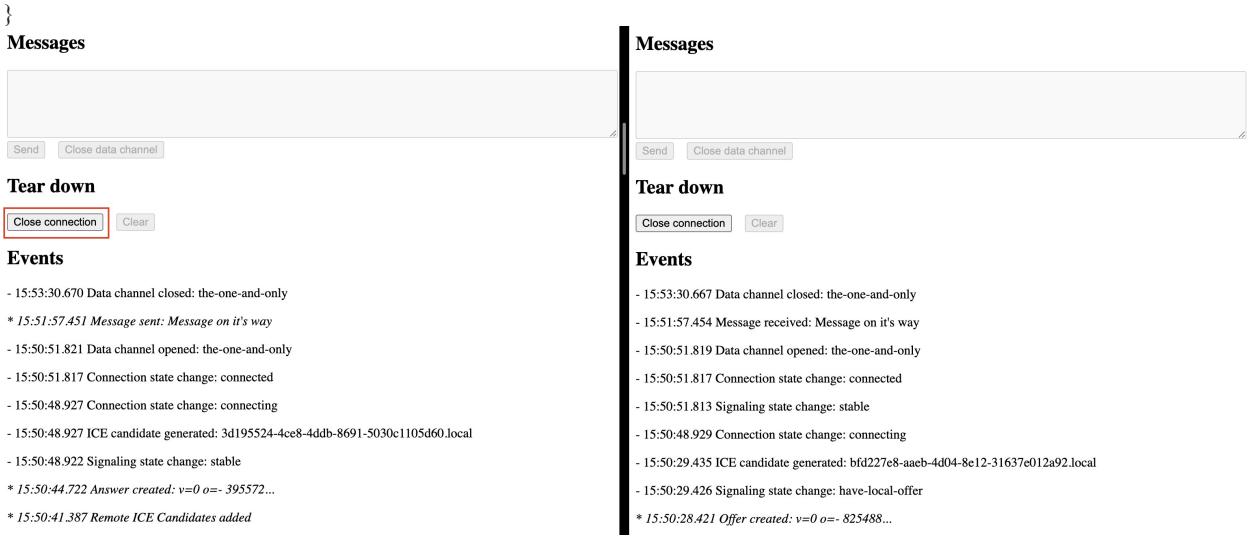
- Tracks trigger `ended` events when "playback or streaming has stopped because the end of the media was reached or because no further data is available". Stopping a track however will not generate an ended event on neither of the peers. In other words, peer A will not get notified when peer B stops consuming peer A's tracks, neither when peer B stops peer B's own tracks.

Connection tear down

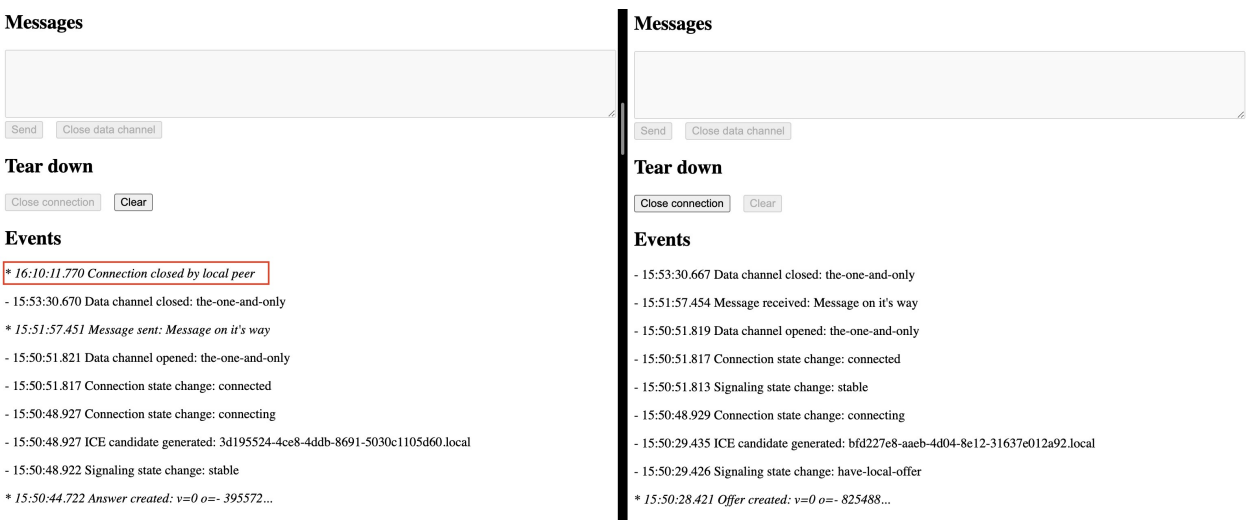
The connection can be terminated by calling the `close` method on the RTCPeerConnection object, which will close any existing data channels and close the connection itself. Note however that closing the connection will not send any "closed" event to the peer. We will need to either use the signaling service to let the peer know that we have terminated the connection or rely on `onConnectionStateChange` (not supported in Firefox) to detect the change of the connection state.

The same applies to media stream tracks and HTML video elements: they both need to be stopped explicitly.

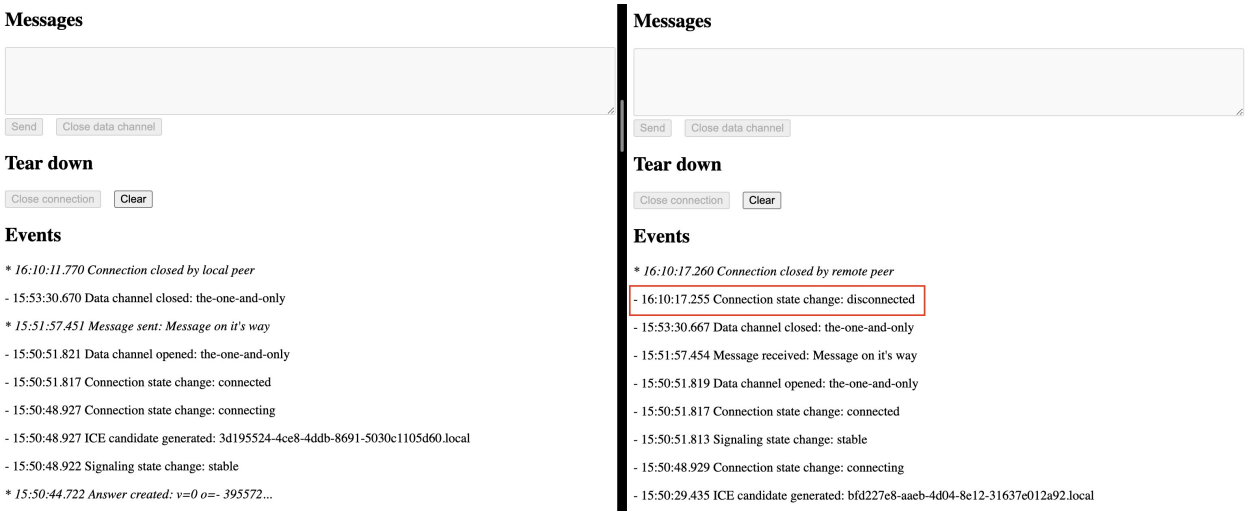
```
export function closeConnection(connection: RTCPeerConnection) {
  connection.close();
}
```



Closing connection



Connection closed in local peer



Connection closed in remote peer

Troubleshooting

Finally, here are a few common error messages you might come across, their meaning and how to fix them.

```
✕ DOMException: Failed to execute 'addIceCandidate' on
'RTCPeerConnection': The remote description was null
```

You are most likely adding remote ICE candidates to a connection object before the remote description has been set. The remote description must be set BEFORE adding ice candidates.

```
✕ Failed to execute 'setLocalDescription' on 'RTCPeerConnection': Failed
to set local offer sdp: Called in wrong state: have-remote-offer
```

You are most likely trying to set an offer as the local description on a connection object that has already set it's remote description. Once the remote description has been set, the connection object can only set an answer as the local description.

```
✕ Failed to execute 'setLocalDescription' on 'RTCPeerConnection': Failed
to set local answer sdp: Called in wrong state: stable
```

You are most likely trying to set the local description on a connection object that has already established a connection.

```
✕ Failed to execute 'createAnswer' on 'RTCPeerConnection':
PeerConnection cannot create an answer in a state other than have-
remote-offer or have-local-pranswer.
```

You are most likely trying to create an answer from a connection object that has not set it's remote description yet. A connection object can only generate an answer AFTER having set a remote description.

❌ Failed to execute 'send' on 'RTCDataChannel': RTCDataChannel.readyState is not 'open'

You are most likely trying to send data through a channel that has not yet emitted its `onopen` event.

❌ Failed to execute 'createOffer' on 'RTCPeerConnection': The RTCPeerConnection's signalingState is 'closed'

You are most likely trying to create an answer from a connection object that has already established and closed a connection. Connection objects can only be used ONCE.

❌ No ICE candidates are generated when setting the local description.

You have most likely created an offer from a connection object before adding media to the connection. Peer A must always add media to the connection (i.e. data channels and/or stream tracks) BEFORE creating an offer and setting the local description.

Wrapping up

And that is all I can tell about WebRTC! You can have a look at this sample repository to find out more about implementation details, but you have everything you need to start fiddling with WeRTC. May the luck be with you and happy coding ⌨

Posts timeline

 Previous
Inferring network requests' return type: @express-typed-api

Following ➡
Distribution of Typescript shared code without package repositories